

DIGITAL FORENSICS LABORATORY · LAB 04

Web Attack Forensics

Working paper — Apache access logs, ModSecurity audits, and the breach timeline

Name	Student ID (NIM)	Class / Group
------	------------------	---------------

Date	Instructor / TA	Session grade
------	-----------------	---------------

1. Lab Overview

This working paper accompanies Lab 04 of the digital forensics series. A web application is attacked, and the evidence is written by the victim itself: Apache records every request, PHP confesses its failures, and ModSecurity keeps a full transcript of requests and responses. You will learn to triage access logs with `awk` and `grep`, read WAF audit transactions section by section, decode layered payloads, and rebuild a complete breach timeline. Complete the pre-lab before the session, check off each step as you perform it, answer every question with quoted log lines, and close with a reflection.

1.1 Learning objectives

- Explain what `access.log`, `error.log`, and `modsec_audit.log` each record, and where they live.
- Parse an Apache combined log line field by field (IP, time, method, URI, status, size, agent).
- Recognize the log signatures of path traversal, remote command execution, and SQL injection.
- Navigate a ModSecurity audit transaction (A/B/C/F/E/H/Z) and correlate it to `access.log` timestamps.
- Interpret OWASP CRS rule IDs and the inbound anomaly score verdict.
- Decode layered payloads: URL encoding (single and double), base64, and SQL `char()` sequences.
- Rebuild an attack timeline and separate attempted attacks from confirmed compromise.

1.2 Environment and files

Environment: Kali Linux (or any Linux VM) with Docker for the vulnerable lab app. The attack analysis itself needs only the log files and standard tools (`grep`, `awk`, `sort`, `uniq`, `python3`). Copy

every evidence file to your working directory first - never edit logs in place, and never expose the lab app beyond localhost.

Table 1.1 — Lab files

File	Purpose
files/logs.zip	Challenge A — access.log, error.log, modsec_audit.log of the breached server (upstream)
files/app/	the deliberately vulnerable Docker app: images/view/command/users pages (upstream)
files/traffic_shop.log	Challenge B (new) — 1,488 lines of shop traffic with one bad actor
files/waf_alert.log	Challenge C (new) — four ModSecurity audit transactions to dissect

1.3 What to submit

- Section 1.4 pre-lab, answered before the session (worth 10 points).
- This working paper as PDF, named NIM_Name_Lab04.pdf.
- Your evidence folder NIM_Lab04/evidence/ containing every numbered exhibit (B-01, B-02, and so on).
- Answers that quote the exact log file and line, field, or rule ID - not vague descriptions.

1.4 Pre-Lab Check

Answer these before the session, using the lab README and Section 2 of this paper as references. One or two sentences per question are enough - precision beats length.

P1. What three log files does the lab environment produce, and what does each one record?

Answer:

Two sentences maximum.

P2. In an Apache combined access log line, what do the status code and response size tell an investigator about an attack request?

Answer:

Two sentences maximum.

P3. Why can access.log never show the payload of a POST request, and where does that payload surface instead?

Answer:

Two sentences maximum.

P4. What does URL encoding do, and why might an attacker encode the same payload twice?

Answer:

Two sentences maximum.

P5. What is a WAF, and what does it mean when ModSecurity runs in DETECTION_ONLY mode?

Answer:

Two sentences maximum.

2. Theory Primer

2.1 The three logs

Apache writes one access.log line per request: client IP, timestamp with timezone, method, URI with query string, protocol, status code, response size in bytes, referer, and user agent. error.log collects what went wrong: PHP warnings with the failing file and line, permission failures, and ModSecurity warnings duplicated in grep-able form. modsec_audit.log is the richest source: for every interesting transaction the WAF stores the full request headers (B), the request body (C), the response headers (F), the response body (E), and an audit trailer (H) listing every rule that matched, all tied together by one transaction id.

Table 2.1 — Field map of a combined access log line

Field	Example	Forensic use
client IP	192.168.0.106	who - rank IPs to find the outlier
timestamp	[16/Feb/2023:01:35:27 +0500]	when - mind the timezone offset
request	"GET /view.php?image=../etc/passwd HTTP/1.1"	what - payloads sit in the URI
status	200 / 302 / 404	outcome class - 404 bursts mean scanning
size	650	how much came back - dumps are big
agent + referer	"Mozilla/5.0 ... Firefox/102.0"	browser fingerprint or tool signature

2.2 The three attack classes and their signatures

Table 2.2 — Attack signatures to memorize

Attack	Looks like	Confirming evidence
Path traversal	../ sequences or %2F encodings in URIs, depth ladder	response contains file content; rules 930100/930110/930120

Attack	Looks like	Confirming evidence
Command execution	POSTs to one endpoint; C-section cmd=, ;, , backticks	E-section echoes command output; rules 932150/932160
SQL injection	quotes + SQL keywords; boolean pairs 1=1 / 1=2; UNION SELECT	E-section shows dumps or SQL errors; rule 942100 libinjection

Automated tools sign their work: sqlmap announces itself in the user agent and probes with random short strings (HOvh, fMgn); dirb and gobuster produce 404 bursts at machine speed. A boolean probe pair (AND 1=1 then AND 1=2) with different response sizes is the classic blind SQL injection test.

2.3 OWASP CRS rule IDs and anomaly scoring

ModSecurity matches requests against the OWASP Core Rule Set. The rule id in every warning is a lookup key: 920xxx protocol anomalies, 930xxx file inclusion and traversal, 932xxx remote code execution, 942xxx SQL injection. Each match adds points to a category; when the total crosses the threshold, rule 980130 prints the verdict line - for example 'Total Inbound Score: 43 - SQLI=0, XSS=0, RFI=0, LFI=35, RCE=5'. One line, full attack profile.

2.4 Decoding layered payloads

URL encoding escapes special characters as %XX (slash %2F, quote %27, equals %3D) and plus signs for spaces; attackers encode twice (%252F decodes to %2F then to /) to slip past naive filters. Base64 blobs may hide underneath, and SQL char(90) sequences are just ASCII codes spelled out. The discipline: decode until the string stops changing - url decode, url decode again, then base64 -d, then read.

3. Guided Exercises

Work through the five drills on the upstream logs.zip (extract to your working directory first and hash the archive). Every output is an exhibit: capture it, number it (B-01, B-02, and so on), and log it in Appendix B.

3.A Access log triage

- ❑ 1. Extract logs.zip; record the MD5 of each of the three log files.
- ❑ 2. Rank client IPs by request count (awk + sort + uniq -c). Which IP stands out?
- ❑ 3. Produce the status-code distribution. What do the 404s cluster around?
- ❑ 4. Rank response sizes descending. Which three requests moved the most bytes?
- ❑ 5. List every distinct user agent. Which one is not a browser?

3.B Reading the audit log

- 1. Count transactions in modsec_audit.log (grep -c "-Z--").
- 2. Pick the transaction nearest 01:37:49 and print it whole with sed -n '/--ID-A--/,/--ID-Z--/p'.
- 3. Identify its C-section (request body) and E-section (response body). What file was read?
- 4. From the H-section: which rule IDs fired, and what does the 980130 line total?

3.C Payload decoding warm-up

- 1. URL-decode: ..%2F..%2F..%2F..%2Fetc%2Fpasswd - show the result.
- 2. URL-decode twice: ctf%257Bd0ubl3%257D - show each layer.
- 3. Base64-decode: Wm94aDN4YW1wbGU= (python3 or base64 -d).
- 4. Decode the SQL sequence char(72)+char(105) with a one-line Python join.

3.D Your own lab (optional, needs Docker)

- 1. Build and run the app (docker build -t app:latest . ; docker run -p 9090:80 app:latest) - localhost only.
- 2. Visit images.php and try one traversal payload yourself; find your own request in access.log.
- 3. Enter id in command.php; find the command in the audit log C-section.
- 4. When finished: docker stop the container. Never leave this app running.

3.E Error log sweep

- 1. grep 'Permission denied' error.log - which file could the attacker NOT read?
- 2. grep 'SQLi' error.log - how many detections, at what times?
- 3. grep -oE '[id "[0-9]+"]' error.log | sort | uniq -c - rank the triggered rules.

4. Challenges

Challenge A is the upstream breach case - solved together in the Deep Dive session. Challenges B and C are yours: work on copies, hash before and after, quote exact lines. Flags use ctf{xxxx} (Challenge A keeps the upstream flag format). Hints are staged - stop after each one and retry.

4.1 Challenge A — the breach case (solved in the Deep Dive)

The ten questions from the lab README: origin IP, exploited vulnerabilities with evidence, the attacker's browser, automated tools, the file that resisted, the data that did not, the compromised secret, the message to the administrator, attack indicators, and indicators of compromise for the future. The Deep Dive walks the full solution - your deliverable is the same answers backed by YOUR exhibits.

4.2 Challenge B — the shop traffic (files/traffic_shop.log)

- 1. Hash the file, then rank client IPs by request count. Who is the outlier, and how many requests do they send?
- 2. Produce the status-code distribution for that IP only. What happened between 07:53 and 07:55?
- 3. Identify the automated tool: quote its user agent string and state its purpose.
- 4. One scanned path returned 200 instead of 404. Which path, and what did the attacker do there minutes later?
- 5. At 07:58 four requests form a classic test pair. Quote both URIs and explain what the size difference proves.
- 6. 08:01 holds two UNION SELECT requests. What data was dumped in each (name the columns), and how do the sizes compare to normal search results?
- 7. Decode the 08:04 'goodbye' query fully and recover the flag.

Hints: (1) `awk '$1=="45.9.148.23"'` isolates the actor. (2) The scanner's user agent contains its own name. (3) The goodbye query decodes TWICE - the first decode gives you %7B braces you last saw in Lab 01; decode again.

Flag B: ctf{_____}

4.3 Challenge C — the WAF night shift (files/waf_alert.log)

- 1. Hash the file. How many transactions does it contain, and which section marks the end of each one?
- 2. For each transaction, note: source IP, method, target path, and whether a request body (C-section) exists.
- 3. Which transaction actually exfiltrated data? Quote the decisive E-section lines.
- 4. Name every CRS rule ID that fired across the file, with one line per rule explaining what it detected.
- 5. Transaction c53d5a29 leaked something different - what, and why is it still useful to the attacker?
- 6. The footer of every H-section says Engine-Mode. What does it mean, and what should it be on a production server?

Hints: (1) Four transactions, but only one E-section contains a key. (2) Rule IDs: look at [id "..."] in the Message lines. (3) DETECTION_ONLY is not a bug in the log - it is a configuration choice.

Flag C: ctf{_____}

5. Analysis Questions

Questions 1-3 condense the original lab's ten questions; questions 4-12 extend the case. Quote the exact log file, line, field, or rule ID in every answer.

1. The breach case (files/logs.zip):

- a. Which IP does the attack originate from, and which browser does the attacker use? Cite the fields.
- b. Which three vulnerability classes were exploited? Give one line of log evidence for each.
- c. Which automated tool was used, and what is its purpose?

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

2. Outcomes of the breach:

- a. Which file did the attacker fail to read, and what stopped them?
- b. What confidential data was exposed, and how much (count the records)?
- c. What was the important secret, and from which file?

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

3. The message and the flag:

- a. What did the attacker's message to the administrator say, in full?
- b. Describe your decoding chain from the raw POST body to the readable flag.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

4. Explain why access.log cannot contain POST payloads, and name the exact audit section where the lab's commands surface.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

5. The traversal ladder shows five requests with increasing ../ depth. Why does an attacker escalate depth one step at a time instead of jumping straight to the deepest payload?

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

6. Rule 942100 reports a libinjection fingerprint like 's&1c'. What is libinjection, and why is a fingerprint more robust than keyword matching?

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

7. The two UNION SELECT requests in Challenge B returned 41,877 and 96,234 bytes while normal searches return about 6,000. Explain how response-size baselines work as a detection signal, and one way an attacker could try to defeat them.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

8. Double URL encoding (%252F) is meant to bypass filters. Explain the two decode steps, and say where in a web stack each decode should happen.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

9. ModSecurity ran in DETECTION_ONLY mode. What would have changed in this incident if it were enforcing? Name one attack that would still have succeeded and justify your answer.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

10. Separate indicators of attack from indicators of compromise in this case: list two of each, with the log line that grounds them.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

11. Why must log analysis be performed on copies? What does re-hashing the logs after analysis prove, and what does it not prove?

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

12. Management asks for one sentence: 'what happened?'. Write it - then write the one sentence you would give the engineering team instead.

Answer:

Write your complete answer, quoting the exact log file and line, field, or rule ID used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. Reflection and Self-Assessment

Summarize your main findings, the skills that clicked, and the obstacles you hit - with how you solved them.

Reflection:

Findings, new skills, obstacles, and how you overcame them.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

Table 6.1 — Self-assessment checklist

I can	Yes	Not yet
Field-parse an access log line and rank IPs, codes, sizes		
Recognize traversal, RCE, and SQLi signatures on sight		
Navigate a ModSecurity audit transaction A through Z		
Correlate audit transactions to access log timestamps		
Decode URL (single/double), base64, and char() layers		
Separate attempted attacks from confirmed compromise		
Build a defensible breach timeline with quoted evidence		

7. Appendices

Appendix A — Tool cheat sheet

Task	Command / tool
Rank client IPs	<code>awk '{print \$1}' access.log sort uniq -c sort -rn</code>
Status distribution	<code>awk '{print \$9}' access.log sort uniq -c</code>

Task	Command / tool
User agent ranking	<code>awk -F'"' '{print \$6}' access.log sort uniq -c sort -rn</code>
Isolate one actor	<code>awk '\$1=="45.9.148.23"' access.log</code>
Biggest responses	<code>awk '{print \$10}' access.log sort -rn head</code>
Print one audit transaction	<code>sed -n '/--TXID-A--/,/--TXID-Z--/p' modsec_audit.log</code>
Rank triggered rules	<code>grep -oE '\[id "[0-9]+"\]' error.log sort uniq -c</code>
URL decode (once or twice)	<code>python3 -c "import urllib.parse as u; print(u.unquote_plus(S))"</code>
Base64 decode	<code>echo 'blob' base64 -d</code>
SQL char() sequence	<code>python3 -c "print(''.join(chr(c) for c in [90,109,120]))"</code>
Shell into the container	<code>docker exec -it \$(docker ps -q) bash</code>

Appendix B — Evidence log

One row per exhibit. Keep numbering continuous across the whole session.

What good evidence looks like — model exhibit:

```
[B-06] 2026-10-14 08:05 traffic_shop.log
awk '$1=="45.9.148.23" && $9==404' traffic_shop.log | wc -l
64
-> captured as evidence/B-06_scanner_burst.png
```

- The command, the file, and the count are fully visible.
- The tool and moment of capture are recorded; the filename says what it shows.
- One exhibit supports one finding.
- Bad evidence: a cropped terminal where the command is scrolled away, or a claim with no line quoted.

No.	Time	Activity / Command	Source file / Tool	Notes
B-01				
B-02				
B-03				
B-04				
B-05				
B-06				

No.	Time	Activity / Command	Source file / Tool	Notes
B-07				
B-08				
B-09				
B-10				

Appendix C — Hash verification form

Fill one row per evidence file: hash when received (start) and after analysis (end). Identical values prove faithful handling.

No	Evidence file	Algorithm	Hash (start)	Hash (end)	Match?
1					Yes / No
2					Yes / No
3					Yes / No
4					Yes / No
5					Yes / No
6					Yes / No

Appendix D — Glossary

Term	Meaning in this lab
access.log	Apache's per-request record: IP, time, method, URI, status, size, referer, agent
error.log	Apache/PHP errors and duplicated WAF warnings
modsec_audit.log	ModSecurity's full transaction transcript, stored in lettered sections
audit sections A-Z	A header, B request headers, C request body, F response headers, E response body, H trailer, Z end
transaction id	the hex id shared by all sections of one audit record
combined log format	the standard Apache access line layout this lab's logs follow
path traversal / LFI	reading files outside the web root with ../; LFI adds execution
RCE	remote command execution - attacker-controlled shell commands on the server
SQL injection	malicious SQL through input fields; boolean pairs test, UNION SELECT dumps
WAF / ModSecurity	web application firewall filtering requests before the app sees them
OWASP CRS	the core rule set; rule id prefixes 920/930/932/942 name the attack family

Term	Meaning in this lab
anomaly score (980130)	sum of rule points per transaction; crossing the threshold prints the verdict
libinjection fingerprint	tokenized shape of an input used to spot SQLi beyond keywords
URL encoding	%XX escapes (and + for space); double encoding adds a second layer
sqlmap / dirb	automated SQL injection tool / web content scanner - both sign the user agent
IOA / IOC	indicator of attack (attempted) vs indicator of compromise (successful)
www-data	the low-privilege account Apache/PHP runs as - why /etc/shadow resisted
DETECTION_ONLY	ModSecurity mode that logs and scores but never blocks
working copy	the copy you analyze; originals stay sealed and hashed

Honesty statement: by submitting this working paper I declare that all answers and evidence are my own work, that original evidence files were never modified, and that I followed the lab's ethics rules.

Student signature: _____ Date: _____

Instructor / TA signature: _____ Grade: _____